Figure 1: DOM inspector

Figure 2: Inspecting content documents

DOM. Different browsers have different implementations of it, which exhibit varying degrees of conformity to the actual DOM standard but every browser uses some DOM to make Web pages accessible to the script.

When you create a script, whether it's inline in a script element or included in the Web page by means of a script loading instruction, you can immediately begin using the API for the document or window elements. This is to manipulate the document itself or to get at the children of that document, which are the various elements in the Web page.

Your DOM programming may be something as simple as the following, which displays an alert message by using the *alert( )* function from a window object or it may use more sophisticated DOM methods to actually create them, as in the longer examples that follow:

```
<body onload = "window.alert ('welcome to my home page!');" >
```

Aside from the script element in which JavaScript is defined, this JavaScript sets a function to run when the document is loaded. This function creates a new element H1, adds text to that element, and then adds H1 to the tree for this document, as shown below:

```
<html>
  <head>
    <script>
   // run this function when the document is loaded
       window.onload = function() {
   // create a couple of elements
   // in an otherwise empty HTML page
       heading = document.createElement("h1");
       heading_text = document.createTextNode("Big Head!");
       heading.appendChild(heading_text);
       document.body.appendChild(heading);
      }
    </script>
  </head>
  <body>
  </body>
</html>
```

DOM interfaces

These interfaces just give you an idea about the actual things that you can use to manipulate the DOM hierarchy. The object representing the *HTMLFormElement* gets its name property from the *HTMLFormElement* interface but its *className* property from the *HTMLElement* interface. In both cases, the property you want is simply in the form object.

Interfaces and objects

Many objects borrow from several different interfaces. The table object, for example, implements a specialised HTML table element interface, which includes such methods as *createCaption* and *insertRow.* Since an HTML element is also, as far as the DOM is concerned, a node in the tree of nodes that makes up the object model for a Web page or an XML page, the table element also implements the more basic node interface, from which the element derives.

When you get a reference to a table object, as in the following example, you routinely use all three of these interfaces interchangeably on the object, perhaps unknowingly:

```
var table = document.getElementById ("table");
var tableAttrs = table.attributes; // Node/Element interface
for (var i = 0; i < tableAttrs.length; i++) {
 // HTMLTableElement interface: border attribute
  if(tableAttrs[i].nodeName.toLowerCase() == "border")
    table.border = "1";
}
// HTMLTableElement interface: summary attribute
table.summary = "note: increased border" ;
```

Core interfaces in the DOM

These are some of the important and most commonly used interfaces in the DOM. These common APIs are used in the longer examples of DOM. You will often see the following APIs, which are types of methods and properties, when you use DOM.

The interfaces of document and window objects are generally used most often in DOM programming. In simple terms, the window object represents something like the browser, and the document object is the root of the document itself. The element inherits from the generic node interface and, together, these two interfaces provide many of the methods and properties you use on individual elements. These elements may also have specific interfaces for dealing with the kind of data those elements hold, as in the table object example.